

PROJECT DOCUMENT / TECHNICAL REPORT

Advanced AI Medical Intelligence Platform (MedIntel)

End-to-end Deep Learning system for medical image analysis,
explainable AI, LLM-assisted reporting, REST APIs, and web deployment

GitHub: <https://github.com/KavyasriLade123/Advanced-AI-Medical-Intelligence>

Document type: Academic / submission project report

Disclaimer: Educational decision support only — not clinical diagnosis

1. Introduction

Medical imaging is central to modern diagnosis, but interpreting scans at scale is time-consuming and expertise-intensive. This project builds an integrated software platform that accepts a medical image, predicts a finding using a deep neural network, explains the prediction visually with Grad-CAM, generates an AI-assisted textual report, exposes results through REST APIs, stores history in a database, and presents everything in a user-friendly React web interface.

The system is designed as an academic demonstration of full-stack AI engineering: data preprocessing, model inference, explainability, generative reporting, API design, persistence, containerization, and frontend UX — not as a certified clinical device.

2. Project Objectives

The platform was built to satisfy the following end-to-end capabilities:

- Analyze uploaded medical images (preprocess and normalize for CNN input).
- Predict findings using Deep Learning (fine-tuned ResNet18 classifier).
- Explain predictions using Explainable AI (Grad-CAM heatmaps).
- Generate AI-assisted medical-style reports using an LLM (with template fallback).
- Provide REST APIs for prediction, history, reports, and health checks.
- Store prediction history in a relational database (SQLite).
- Deploy with a user-friendly interface (React) and optional Docker Compose.

3. Scope and Supported Findings

The trained classifier supports multi-region medical image triage with the following labels:

- Chest: NORMAL, PNEUMONIA
- Brain: BRAIN_NORMAL, BRAIN_TUMOR
- Breast: BREAST_NORMAL, BREAST_MALIGNANT
- Other: ABDOMEN, BONE_FRACTURE, EYE_RETINA, LOWER_LIMB, SKIN
- Reject class: UNSUPPORTED (plus low-confidence rejection messaging)

Unrelated or low-confidence uploads are rejected with a clear message asking the user to upload a correct medical image from supported body regions.

4. System Architecture

MedIntel follows a client–server architecture. The React frontend communicates with a FastAPI backend. The backend loads a PyTorch ResNet18 checkpoint, runs inference and Grad-CAM, optionally calls an OpenAI-compatible LLM for report text, and persists records in SQLite via SQLAlchemy.

4.1 High-level flow

- User opens the web UI (Home / About / Analyze).
- User uploads a JPG/PNG medical image on the Analyze page.
- Frontend POSTs the file to **/api/predict**.
- Backend preprocesses the image (RGB, 224×224, ImageNet normalize).
- ResNet18 predicts class probabilities; Grad-CAM produces a heatmap overlay.
- Disease info card and LLM/template report are assembled for the top finding.
- Result is stored in SQLite and returned as JSON (prediction, confidence, heatmap URL, report).
- UI displays only the predicted finding (not a full class bar chart).

4.2 Technology stack

Layer	Technologies
Frontend	React, TypeScript, Vite
Backend	FastAPI, Uvicorn, Pydantic, SQLAlchemy
Deep Learning	PyTorch, Torchvision (ResNet18)
Explainability	Grad-CAM on ResNet layer4
LLM reports	OpenAI-compatible chat API + template fallback
Database	SQLite
Packaging	Docker, Docker Compose, nginx (frontend image)

5. Deep Learning Model

The classifier is a ResNet18 network pretrained on ImageNet, with a replaced fully connected head for the project's multi-class medical labels. Weights are stored at **backend/models/chest_xray_resnet18.pth**.

5.1 Training approach

- Input: RGB images resized to 224×224 with ImageNet mean/std normalization.
- Loss: class-weighted CrossEntropy to mitigate class imbalance.
- Optimizer: Adam.
- Pipeline: head-only warm-up followed by full fine-tuning.
- Training endpoint: **python -m app.ml.train** (see backend ML module).

5.2 Inference safeguards

A minimum confidence threshold (default 0.50) and an UNSUPPORTED class help reject non-medical or out-of-distribution images. This improves UX honesty and reduces misleading high-confidence errors on unrelated photos.

6. Explainable AI (Grad-CAM)

Grad-CAM computes class-discriminative localization maps from gradients flowing into the final convolutional block (ResNet **layer4**). The heatmap is overlaid on the original image and returned to the client so users can see which regions most influenced the model's decision. This supports transparency and educational review of model focus.

7. AI-Assisted Medical Reports

After prediction, the backend builds a structured disease-information card for the predicted label and generates a narrative report. If an OpenAI-compatible API key is configured, the system requests a concise clinical-style summary from the LLM. If the key is missing or the call fails, a deterministic template report is returned so the demo remains fully functional offline.

8. REST API Design

APIs are served under the **/api** prefix. Interactive OpenAPI docs are available at **/docs** when the backend is running.

Endpoint	Purpose
/api/health	Service health / readiness check
/api/predict	Upload image → prediction, Grad-CAM, report
/api/history	List stored prediction records
/api/reports	Report-related retrieval / generation helpers

9. Database and Persistence

Prediction history is stored in SQLite using SQLAlchemy. Typical fields include timestamp, predicted class, confidence, image path, heatmap path, and report text. This enables auditability of demo sessions and a history view in the UI/API.

10. Frontend Application

The frontend is a React + TypeScript SPA built with Vite and React Router. Pages include:

- bullet **Home (/)** — product introduction and call-to-action to analyze images.
- bullet **About (/about)** — project purpose, capabilities, and disclaimer.
- bullet **Analyze (/analyze)** — image upload, prediction result, Grad-CAM, report.

Development servers can bind to **0.0.0.0** so the UI is reachable on the local network (e.g., `http://192.168.x.x:5173`). Vite proxies **/api** to the FastAPI backend on port 8000.

11. Deployment and Docker

Containerization is implemented for reproducible deployment:

- bullet **backend/Dockerfile** — Python 3.11 image running Uvicorn on port 8000.
- bullet **frontend/Dockerfile** — multi-stage build serving static assets via nginx.
- bullet **docker-compose.yml** — orchestrates backend and frontend together.

Public cloud live deployment is optional and may be added later (Render, Railway, Azure, etc.). Local and LAN demos satisfy academic evaluation when Docker/cloud is unavailable.

12. Repository Structure (Summary)

- bullet **backend/app/** — FastAPI app, ML, routes, services, schemas, database.
- bullet **backend/models/** — trained PyTorch weights (.pth).
- bullet **frontend/src/** — React pages and components.
- bullet **docs/** — this PDF, model card, submission notes.
- bullet **scripts/** — helpers including this PDF generator.
- bullet **requirements.txt** — root wrapper installing backend dependencies.

13. Submission Deliverables

Deliverable	Location / Link
Complete source code	GitHub repository (backend + frontend)
Trained model	backend/models/chest_xray_resnet18.pth
GitHub link	github.com/KavyasriLade123/Advanced-AI-Medical-Intelligence
README	README.md
PDF project report	docs/Project_Report.pdf (this document)
requirements.txt	requirements.txt + backend/requirements.txt
Dockerfile	backend/Dockerfile, frontend/Dockerfile, compose
Live deployment	Optional — local/LAN demo if not cloud-hosted

14. Results and Observations

The integrated pipeline successfully performs upload → inference → Grad-CAM → report → history in a single user flow. Multi-class expansion beyond binary chest pneumonia improves coverage across common imaging domains used in demos. Rejection of unsupported images reduces false clinical-sounding outputs on random photographs.

Limitations include dependence on training data quality/diversity, possible confusion between visually similar modalities, and the fact that template/LLM text is assistive narrative rather than a radiologist report. Performance metrics should be re-evaluated whenever the class set or dataset is updated.

15. Ethics, Safety, and Disclaimer

This project is intended strictly for education, research demonstration, and software engineering evaluation. It must not be used for real patient diagnosis, triage decisions, or clinical treatment planning. Any LLM-generated text may contain inaccuracies and must be reviewed by qualified professionals in

any real-world medical context.

16. Conclusion and Future Work

MedIntel demonstrates a complete Advanced AI Medical Intelligence stack: deep learning prediction, visual explainability, generative reporting, APIs, persistence, and a modern web UI with Docker packaging. Future improvements may include larger curated multi-modal datasets, stronger calibration, DICOM support, role-based auth, cloud deployment with HTTPS, and formal clinical validation studies.

— End of Project Document —

Generated for submission · Advanced AI Medical Intelligence Platform (MedIntel)